LAPORAN PRAKTIKUM
STRUKTUR DATA

MODUL 3
ABSTRACT DATA TYPE (ADT)



Disusun Oleh :
NAMA : Daffa Asqalany
NIM: 109102500013

Dosen:
Diah Septiani, S.Kom., M.C.S. (AI). M.Kom.

PROGRAM STUDI S-1 SAINS DATA
FAKULTAS INFORMATIKA
UNIVERSITAS TELKOM
2026

A. Konsep dan Definisi Abstract Data Type (ADT)

Abstract Data Type (ADT) atau Tipe Data Abstrak merupakan sebuah model matematika yang mendefinisikan suatu tipe data bersama dengan sekumpulan operasi primitif yang dapat dieksekusi terhadap tipe data tersebut. Secara konseptual, ADT menekankan prinsip abstraksi data dan enkapsulasi, di mana perilaku fungsional dari tipe data dipisahkan secara tegas dari representasi fisik dan detail implementasi internalnya di dalam memori komputer. Hal ini memungkinkan pemrogram untuk fokus pada cara entitas data berinteraksi dan diproses tanpa harus terbebani oleh rincian teknis struktur data di tingkat memori tingkat rendah.

Dalam sebuah ADT yang lengkap, pendefinisian mencakup invarian tipe (batasan keadaan data yang valid) serta aksioma yang berlaku. Elemen-elemen operasi primitif di dalam ADT dikelompokkan ke dalam beberapa kategori fungsional, yaitu:

1. Konstruktor/Kreator (Creator): Operasi pembentuk nilai tipe data atau instansiasi objek, biasanya diawali dengan predikat create atau make.
2. Selektor (Getter): Operasi yang digunakan untuk membaca atau mengakses nilai dari komponen spesifik suatu tipe data.
3. Mutator (Setter): Prosedur yang bertugas memodifikasi atau memperbarui nilai dari komponen spesifik tipe data.
4. Validator: Operasi logika yang mengevaluasi apakah instansiasi nilai suatu tipe memenuhi aturan batasan atau domain yang ditentukan.
5. Destruktor/Dealokator: Operasi untuk membebaskan ruang memori yang dialokasikan untuk suatu entitas data saat sudah tidak lagi digunakan.
6. Fungsi Baca/Tulis: Komponen antarmuka yang menangani aliran masukan (input) dan keluaran (output) pada perangkat.
7. Operator Relasional dan Aritmatika: Operasi komparasi relasional (seperti kesetaraan atau perbandingan besar-kecil) serta operasi perhitungan matematis yang didefinisikan secara khusus untuk tipe bentukan tersebut.

B. Struktur Modular dan Implementasi ADT dalam Pemrograman C++

Dalam paradigma prosedural dan pemrograman modular menggunakan bahasa C++, konsep ADT diterjemahkan melalui pembagian arsitektur kode ke dalam tiga berkas (file) terpisah:

1. Berkas Header (.h): Berisi spesifikasi tipe bentukan (struct) dan prototipe primitif (deklarasi fungsi dan prosedur). Berkas ini bertindak sebagai kontrak antarmuka (interface). Di dalamnya disematkan preprocessor wrapper/guards (#ifndef, #define, #endif) untuk menghindari kesalahan deklarasi ganda saat proses kompilasi.
2. Berkas Realisasi/Implementasi (.cpp): Memuat tubuh atau definisi lengkap (body) dari masing-masing fungsi dan prosedur primitif yang telah dideklarasikan pada berkas header. Realisasi ini mengoptimalkan penggunaan konstruktor dan selektor untuk menjamin integritas manipulasi data.
3. Berkas Driver/Program Utama (main.cpp): Berisi alur logika eksekusi utama program yang menyertakan (include) berkas header ADT. Berkas ini berfungsi untuk menguji instansiasi data serta pemanggilan operasi-operasi primitif yang telah dibangun.

Penerapan modularitas ini tidak hanya meningkatkan keterbacaan (readability) dan modularitas program, tetapi juga memaksimalkan reusability, di mana pustaka ADT yang telah dirancang dapat diintegrasikan ke proyek lain tanpa mengubah implementasi dasarnya.

Latihan Modul 3

Tugas 1

Input:

```cpp
#include <iostream>
#include <iomanip>
#include <string>

using namespace std;

struct Mahasiswa {
    string nama;
    string nim;
    float uts;
    float uas;
    float tugas;
    float nilaiAkhir;
};

float hitungNilaiAkhir(float uts, float uas, float tugas) {
    return (0.3f * uts) + (0.4f * uas) + (0.3f * tugas);
}

int main() {
    const int MAX_MHS = 10;
    Mahasiswa daftarMhs[MAX_MHS];
    int n;

    cout << "========================================" << endl;
    cout << "     PROGRAM PENGELOLAAN NILAI MAHASISWA    " << endl;
    cout << "========================================" << endl;
    cout << "Masukkan jumlah mahasiswa (maksimal 10): ";
    cin >> n;

    if (n > MAX_MHS || n <= 0) {
        cout << "Jumlah data tidak valid! Program berhenti." << endl;
        return 1;
    }

    for (int i = 0; i < n; i++) {
        cout << "\nData Mahasiswa ke-" << (i + 1) << ":" << endl;
        cout << "Nama        : ";
        cin.ignore();
        getline(cin, daftarMhs[i].nama);
        cout << "NIM         : ";
        cin >> daftarMhs[i].nim;
        cout << "Nilai UTS   : ";
        cin >> daftarMhs[i].uts;
        cout << "Nilai UAS   : ";
        cin >> daftarMhs[i].uas;
        cout << "Nilai Tugas : ";
        cin >> daftarMhs[i].tugas;

        daftarMhs[i].nilaiAkhir = hitungNilaiAkhir(
            daftarMhs[i].uts,
            daftarMhs[i].uas,
            daftarMhs[i].tugas
        );
    }

    cout << "\n======================================================================" << endl;
    cout << left << setw(5)  << "No"
         << setw(20) << "Nama"
         << setw(15) << "NIM"
         << setw(8)  << "UTS"
         << setw(8)  << "UAS"
         << setw(8)  << "Tugas"
         << setw(10) << "Akhir" << endl;
    cout << "======================================================================" << endl;

    for (int i = 0; i < n; i++) {
        cout << left << setw(5)  << (i + 1)
             << setw(20) << daftarMhs[i].nama
             << setw(15) << daftarMhs[i].nim
             << fixed << setprecision(2)
             << setw(8)  << daftarMhs[i].uts
             << setw(8)  << daftarMhs[i].uas
             << setw(8)  << daftarMhs[i].tugas
             << setw(10) << daftarMhs[i].nilaiAkhir << endl;
    }
    cout << "======================================================================" << endl;

    return 0;
}
```

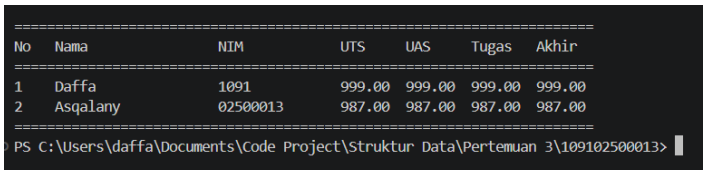
Input:

```
========================================
     PROGRAM PENGELOLAAN NILAI MAHASISWA
========================================
Masukkan jumlah mahasiswa (maksimal 10): 2

Data Mahasiswa ke-1:
Nama        : Daffa
NIM         : 1091
Nilai UTS   : 999
Nilai UAS   : 999
Nilai Tugas : 999

Data Mahasiswa ke-2:
Nama        : Asqalany
NIM         : 02500013
Nilai UTS   : 987
Nilai UAS   : 987
Nilai Tugas : 987
```

Output:



Deskripsi:

Program ini dirancang untuk mendemonstrasikan konsep pengelompokan data majemuk menggunakan tipe data bentukan struct yang dikombinasikan dengan larik statis (array). Entitas Mahasiswa dideklarasikan dengan sejumlah atribut anggota (field), meliputi identitas berupa nama dan NIM berformat teks, serta nilai UTS, UAS, Tugas, dan Nilai Akhir bertipe bilangan pecahan desimal (float).

Pada fase pemrosesan, program meminta masukan variabel jumlah mahasiswa dengan batasan maksimum 10 elemen. Struktur iterasi perulangan for digunakan untuk menerima masukan data setiap mahasiswa secara sekuensial. Nilai akhir mahasiswa tidak dihitung secara manual di dalam blok utama, melainkan didelegasikan kepada subprogram fungsi hitungNilaiAkhir. Fungsi ini mengimplementasikan pembobotan nilai yaitu 30% untuk UTS, 40% untuk UAS, dan 30% untuk Tugas melalui operasi perkalian floating point. Hasil kembalian (return value) fungsi tersebut kemudian dialokasikan ke dalam field nilaiAkhir pada indeks elemen yang bersangkutan. Pada tahap terminasi, program memanfaatkan manipulator tabel dari pustaka <iomanip> (setw, left, fixed, setprecision) untuk memformat dan mencetak rekapitulasi data mahasiswa secara rapi ke konsol.

Tugas 2

```h
#ifndef PELAJARAN_H_INCLUDED
#define PELAJARAN_H_INCLUDED

#include <iostream>
#include <string>

using namespace std;

struct pelajaran {
    string namamapel;
    string kodemapel;
};

pelajaran create_pelajaran(string namapel, string kodepel);

void tampil_pelajaran(pelajaran pel);

#endif
```

```cpp
#include "pelajaran.h"

pelajaran create_pelajaran(string namapel, string kodepel) {
    pelajaran pel;
    pel.namamapel = namapel;
    pel.kodemapel = kodepel;
    return pel;
}

void tampil_pelajaran(pelajaran pel) {
    cout << "nama pelajaran: " << pel.namamapel << endl;
    cout << "nilai: " << pel.kodemapel << endl;
}
```

```cpp
#include <iostream>
#include "pelajaran.h"

using namespace std;

int main() {
    string namapel = "Struktur Data";
    string kodepel = "STD";

    pelajaran pel = create_pelajaran(namapel, kodepel);
    tampil_pelajaran(pel);

    return 0;
}
```

Screenshots Input

(Tidak perlu input)

Screenshots Output

```
PS C:\Users\daffa\Documents\Code Project\Struktur Data\Pertemuan 3\Soal 2> g++ main.cpp pelajaran.cpp -o main.ex
PS C:\Users\daffa\Documents\Code Project\Struktur Data\Pertemuan 3\Soal 2> .\main.exe
nama pelajaran: Struktur Data
nilai: STD
PS C:\Users\daffa\Documents\Code Project\Struktur Data\Pertemuan 3\Soal 2>
```

Deskripsi:

Program ini merepresentasikan penerapan arsitektur pemrograman modular berbasis Tipe Data Abstrak (ADT) yang terbagi menjadi tiga berkas komponen: berkas deklarasi antarmuka (pelajaran.h), berkas realisasi primitif (pelajaran.cpp), dan pengemudi utama (main.cpp).

Pada berkas pelajaran.h, struktur pelajaran dirumuskan dengan dua anggota data bertipe string, yaitu namamapel dan kodemapel. Di samping itu, dideklarasikan prototipe dua fungsi primitif: create_pelajaran yang bertindak sebagai fungsi kreator/konstruktor, serta tampil_pelajaran yang berperan sebagai prosedur antarmuka cetak (display). Pada berkas pelajaran.cpp, logika tubuh fungsi diwujudkan di mana create_pelajaran menginisialisasi atribut data ke dalam sebuah variabel lokal bentukan dan mengembalikannya ke pemanggil, sedangkan tampil_pelajaran mengekstrak isi struktur untuk dicetak ke konsol sesuai format panduan modul. Pada main.cpp, fungsi main menguji alur tersebut dengan mengirimkan argumen "Struktur Data" dan "STD", yang berhasil menghasilkan instansiasi objek dan mencetak teks keluaran secara presisi.

Tugas 3

```cpp
#include <iostream>
#include <iomanip>

using namespace std;

const int BARIS = 3;
const int KOLOM = 3;

void tampilkanMatriks(int mat[BARIS][KOLOM], string namaMatriks) {
    cout << "Matriks " << namaMatriks << ":" << endl;
    for (int i = 0; i < BARIS; i++) {
        for (int j = 0; j < KOLOM; j++) {
            cout << setw(4) << mat[i][j];
        }
        cout << endl;
    }
}

void tukarElemenMatriks(int matA[BARIS][KOLOM], int matB[BARIS][KOLOM], int b, int k) {
    if (b >= 0 && b < BARIS && k >= 0 && k < KOLOM) {
        int temp = matA[b][k];
        matA[b][k] = matB[b][k];
        matB[b][k] = temp;
        cout << ">> Pertukaran elemen posisi [" << b << "][" << k << "] berhasil dilakukan.\n";
    } else {
        cout << ">> Indeks posisi di luar batas matriks!\n";
    }
}

void tukarNilaiPointer(int *p1, int *p2) {
    if (p1 != nullptr && p2 != nullptr) {
        int temp = *p1;
        *p1 = *p2;
        *p2 = temp;
        cout << ">> Pertukaran nilai via pointer berhasil dilakukan.\n";
    }
}
```

```cpp
int main() {
    int matriks1[BARIS][KOLOM] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };

    int matriks2[BARIS][KOLOM] = {
        {10, 20, 30},
        {40, 50, 60},
        {70, 80, 90}
    };

    cout << "=== KONDISI AWAL MATRIKS 2D ===" << endl;
    tampilkanMatriks(matriks1, "A");
    cout << endl;
    tampilkanMatriks(matriks2, "B");
    cout << endl;

    // 2. Menukarkan elemen array 2D pada posisi baris ke-1, kolom ke-1 (tengah: 5 dan 50)
    int targetBaris = 1;
    int targetKolom = 1;
    cout << "--- Menukarkan elemen koordinat [" << targetBaris << "][" << targetKolom << "] ---" << endl;
    tukarElemenMatriks(matriks1, matriks2, targetBaris, targetKolom);
    cout << endl;

    cout << "=== KONDISI MATRIKS SETELAH PERTUKARAN ELEMEN ===" << endl;
    tampilkanMatriks(matriks1, "A");
    cout << endl;
    tampilkanMatriks(matriks2, "B");
    cout << endl;
```

```cpp
    int valX = 100;
    int valY = 999;
    int *ptr1 = &valX;
    int *ptr2 = &valY;

    cout << "=== PERTUKARAN NILAI DENGAN 2 POINTER ===" << endl;
    cout << "Kondisi Sebelum Tukar:" << endl;
    cout << "Nilai valX       : " << valX << " (ditunjuk oleh *ptr1: " << *ptr1 << ")" << endl;
    cout << "Nilai valY       : " << valY << " (ditunjuk oleh *ptr2: " << *ptr2 << ")" << endl;
    cout << endl;

    tukarNilaiPointer(ptr1, ptr2);
    cout << endl;

    cout << "Kondisi Setelah Tukar:" << endl;
    cout << "Nilai valX       : " << valX << " (ditunjuk oleh *ptr1: " << *ptr1 << ")" << endl;
    cout << "Nilai valY       : " << valY << " (ditunjuk oleh *ptr2: " << *ptr2 << ")" << endl;

    return 0;
}
```

Screenshots Input

(Tidak perlu input)

Screenshots Output

```
=== KONDISI AWAL MATRIKS 2D ===
Matriks A:
   1   2   3
   4   5   6
   7   8   9

Matriks B:
  10  20  30
  40  50  60
  70  80  90

--- Menukarkan elemen koordinat [1][1] ---
>> Pertukaran elemen posisi [1][1] berhasil dilakukan.

=== KONDISI MATRIKS SETELAH PERTUKARAN ELEMEN ===
Matriks A:
   1   2   3
   4  50   6
   7   8   9

Matriks B:
  10  20  30
  40   5  60
  70  80  90

=== PERTUKARAN NILAI DENGAN 2 POINTER ===
Kondisi Sebelum Tukar:
Nilai valX       : 100 (ditunjuk oleh *ptr1: 100)
Nilai valY       : 999 (ditunjuk oleh *ptr2: 999)

>> Pertukaran nilai via pointer berhasil dilakukan.

Kondisi Setelah Tukar:
Nilai valX       : 999 (ditunjuk oleh *ptr1: 999)
Nilai valY       : 100 (ditunjuk oleh *ptr2: 100)
PS C:\Users\daffa\Documents\Code Project\Struktur Data\Pertemuan 3\Soal 3>
```

Deskripsi:

Program ini mendemonstrasikan manipulasi alokasi array multidimensi (matriks 2D) serta pengaksesan alamat memori menggunakan variabel penunjuk (pointer). Program mendeklarasikan dua buah array dua dimensi berordo 3×3, yakni matriks1 dan matriks2, serta dua variabel pointer bertipe integer ptr1 dan ptr2.

Fasilitas penelusuran matriks diwadahi oleh prosedur tampilkanMatriks yang menggunakan perulangan tersarang (nested loop) untuk mencetak baris dan kolom secara rapi. Prosedur tukarElemenMatriks menerima referensi kedua matriks beserta parameter indeks baris dan kolom yang dituju. Di dalam prosedur ini, algoritma penukaran dilakukan menggunakan variabel perantara temp, sehingga nilai elemen matriks pertama pada indeks [b][k] ditukar secara langsung dengan nilai matriks kedua pada posisi yang sama (dibuktikan dengan berubahnya elemen tengah dari angka 5 menjadi 50 pada matriks A, dan sebaliknya pada matriks B).

Selanjutnya, prosedur tukarNilaiPointer mengimplementasikan teknik call-by-pointer. Prosedur menerima parameter formal berupa alamat memori (int *p1, int *p2). Operasi dereference menggunakan operator asterisk (*) dilakukan untuk mengakses dan menukar isi nilai data asli dari variabel valX dan valY yang bernaung di bawah alamat yang ditunjuk oleh masing-masing pointer. Hasilnya menunjukkan bahwa modifikasi nilai pada lokasi memori melalui dereferencing secara langsung memperbarui nilai variabel aktual di luar fungsi.